

UT5. Programación orientada a objetos en PHP

DESARROLLO WEB EN ENTORNO SERVIDOR (DWS)



Contenidos

1. Orientación a objetos en PHP.

- Características de orientación a objetos en PHP.
- Creación de clases.
- Utilización de objetos.
- Mecanismos de mantenimiento del estado.
- Herencia.
- Interfaces.
- Ejemplo de POO en PHP.

2. Programación en capas.

- Namespaces en PHP.
- Gestionar dependencias.
- Separación de la lógica de negocio.
- Generación del interface de usuario.

Programación orientada a objetos

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

Programación orientada a objetos.

Características.



Definiciones:

- La programación orientada a objetos (POO u OOP) es una metodología de programación basada en objetos.
- Un objeto es una estructura que contiene datos (atributos – en PHP se llaman propiedades) y el código (métodos) que los maneja.
- La creación de un objeto basado en una clase se le llama instanciar una clase y al objeto obtenido se le conoce como instancia de esa clase.
- Los pilares fundamentales de la POO son:
 - Herencia: Se crea una clase a partir de otra, heredando comportamiento y características, que se pueden redefinir.
 - Abstracción: Una clase puede presentar una parte oculta gracias a la definición de métodos privados y una parte pública que presenta al objeto como un ente abstracto con la capacidad para realizar un trabajo definido.
 - Polimorfismo: Un mismo método puede tener comportamientos diferentes en función del objeto que lo utilice.
 - Encapsulación: Se unen los datos y el código que los manipula.



Programación orientada a objetos. Características.



Definiciones:

- Las ventajas más importantes de la POO son:
 - Modularidad: debido a que permite dividir el programa en partes o módulos pequeños independientes y bien comunicados entre sí.
 - Extensibilidad: Nuevas características implican añadir nuevos métodos o crear objetos que extiendan el comportamiento de los existentes.
 - Mantenimiento: Son sencillos de mantener gracias a las reglas existentes a la hora de codificar y a su modularidad.
- Actualmente PHP tiene soporte completo para la programación orientada a objetos. Permite definir clases, herencia, interfaces y los elementos habituales.

Programación orientada a objetos.

Creación de clases.

Para declarar una clase se utiliza la palabra reservada **class**. Los atributos se declaran utilizando su nombre, los modificadores que pueda tener y opcionalmente un valor por defecto:

```
class MiClase {  
    private $att1 = 10;           // con valor por defecto  
    private $att2;                // sin valor por defecto  
    private static $att3 = 0;    // estático  
}
```

Los atributos y métodos se pueden declarar como estáticos, de manera que no habrá uno por objeto, sino por clase (static).

Programación orientada a objetos.

Creación de clases.

Se puede crear un constructor **__construct()** éste método es un “método mágico” (nombre reservado a tareas concretas y que comienzan por dos guiones bajos “__”).

Para crear un nuevo objeto se utiliza el operador **new** seguido del nombre de la clase:

```
$obj = new MiClase();
```

Para las propiedades y métodos se pueden utilizar los siguientes modificadores de visibilidad (por defecto serán públicas):

- **public**: Se pueden utilizar desde dentro y fuera de la clase.
- **private**: Pueden emplearse desde la propia clase.
- **protected**: Se pueden utilizar dentro de la propia clase, las derivadas y las antecesoras.

Normalmente las propiedades se declaran como privados y se crean métodos públicos para acceder a ellos.

```
$objeto ->propiedad;           // propiedades
```

```
$objeto ->método(argumentos);  // métodos
```



Programación orientada a objetos.

Creación de clases.

Ejemplo de uso de los diferentes tipos de propiedades **MyClass.php**:

- Creamos dos clases MyClass y una clase MyClass2 que extiende la MyClass.
- **MyClass**: define propiedades de tipo public, protected y private.
 - Se crea un objeto \$obj de la clase MyClass y se trata de acceder a las propiedades.
 - Se pueden acceder a las propiedades públicas pero no a las protected o private.
- **MyClass2**: extiende la clase MyClass y redefine las propiedades publica y protected.
 - Se crea un objeto \$obj2 de la MyClass2 y se trata de acceder a las propiedades definidas.
 - Se pueden acceder a las propiedades públicas pero no a las protected o private (en este caso indefinido por venir de la clase MyClass).



Programación orientada a objetos.

Creación de clases.

Los métodos son funciones definidas dentro de una clase.

Los métodos no estáticos se puede utilizar la palabra reservada **this** que representa el objeto desde el que se invoca el método.

En el fichero **PersonayCliente.php** se verá como se crea una clase.

- Se declara la clase Persona, con los atributos (nombre, apellidos y DNI).
- Se declara su constructor.
- Se declaran los getters y setters necesarios.
- Se declaran dos objetos persona y cliente.



Programación orientada a objetos.

Utilización de Objetos.

En el fichero **PersonayCliente.php** se verá como se crea una clase.

- Se declaran dos objetos persona y cliente.

// Creación de Persona

```
$per = new Persona("1111111A", "Ana", "Puertas");  
echo $per . "<br>";           // mostrarla, usa el método __toString()  
$per->setApellido("Montes");  // cambiar el apellido  
echo $per . "<br>";           // volver a mostrar
```

// Creación de un Cliente (Persona y saldo)

```
$cli = new Cliente("22222245A", "Pedro", "Sales", 100);  
echo $cli . "<br>";           // lo muestra, usa el método __toString()
```



Programación orientada a objetos.

Utilización de Objetos.

El operador de resolución de ámbito (::)

- Proporciona un medio para acceder a los miembros de una constante, una propiedad estática o un método estático de una clase o de una clase de sus padres.
- Es posible referenciar una clase utilizando una variable (no puede ser una palabra clave tipo self, parent y static).
- Veremos en el ejemplo **resolucionAmbito.php**
 - Un ejemplo fuera de la definición de la clase.
 - Un ejemplo desde la definición de la clase.



Programación orientada a objetos.

Mantenimiento de estados.

Para almacenar objetos en las sesiones abiertas por un usuario, éstos se deben serializar.

La función **serialize()** devuelve un string que contiene una representación lineal de cualquier valor que puede ser almacenado en PHP.

La función **unserialize()** puede utilizar ese string para recrear el valor original de la variable a partir de su representación lineal.

Si una aplicación serializa objetos se recomienda que la aplicación incluya las definiciones de las clases de los objetos serializados en cada página. No hacerlo podría resultar en un objeto deserializado sin su definición de clase (clase tipo **__PHP_Incomplete_Class_Name**).

Veamos un ejemplo **serialize.php**.

Fuente: <https://www.php.net/manual/es/language.oop5.serialization.php>



Programación orientada a objetos.

Herencia.

La herencia es uno de los principios de la POO y PHP la implementa en su modelo de objetos.

Cuando una clase es extendida, la clase hija hereda todos los métodos públicos y protegidos, propiedades y constantes de la clase padre.

Los métodos son conservados mientras una clase hija no los sobrescriba.

Los métodos privados de una clase padre no son accesibles para la clase hija. Por ello, las clases hijas deben implementar un método privado por sí mismas.

La visibilidad de los métodos, propiedades y constantes puede ser relajada.



Programación orientada a objetos.

Herencia.

Veremos un ejemplo de herencia **Vecino.php** y **Presidente.php** donde:

- Se tiene una clase Vecino (sólo ve sus datos).
- Se tiene una clase Presidente (ve todos los vecinos y puede modificar una cuota).
- El Vecino tiene las propiedades de: nombre, apellidos, dni, piso, cuota.
- El Presidente es un vecino (es una clase extendida de vecino).
- El fichero **VecinoPresidente.php** tiene ejemplo de uso de las clases.



Programación orientada a objetos.

Interfaces.

Una interface de objetos permite crear código que especifica qué métodos y propiedades una clase debe implementar sin tener que definir la implementación de éstos métodos y propiedades.

Las interfaces comparten el mismo espacio de nombres que las clases, traits y enumeraciones, de modo que una vez definidas, su nombre no puede ser usado.

Los métodos declarados en una interfaz deben ser públicos.

Para definir una interface se utiliza la palabra reservada **interface**.

Para implementar una interfaz, se utiliza el operador **implements**.

Las interfaces también se pueden extender como las clases, utilizando el operador **extends**.

Veremos un ejemplo en **interfaz.php**



Programación orientada a objetos. Traits.

Traits es la manera que ha desarrollado PHP para la reutilización de código en un lenguaje de herencia simple.

El objetivo de **Traits** es reducir la limitación existente de la herencia, permitiendo la reutilización de un cierto número de métodos en clases independientes.

Un **trait** es similar a una clase, pero sólo sirve para agrupar funcionalidades. Permite el uso de métodos de clase sin necesidad de la herencia.

Fuente: <https://www.php.net/manual/es/language.oop5.traits.php>



Programación en capas

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

Programación en capas

La programación en capas ayuda a separar las distintas partes de una aplicación.

Existen varios métodos que permiten separar la presentación de la lógica de negocio. El más extendido es el MVC (Modelo Vista Controlador). Este patrón pretende dividir el código en tres partes:

- **Modelo:** Encargado de manejar los datos propios de la aplicación.
- **Vista:** Encargada de la interacción con el usuario (es la correspondiente a la interfaz con el usuario).
- **Controlador:** Encargada de decidir que se realizará a razón de las acciones que el usuario selecciona en la vista.

Programación en capas.

Namespaces

Namespaces: Representan un medio para encapsular elementos, es decir, es una forma de organizar y evitar conflictos entre nombres de clases, interfaces, funciones o constantes que pueden tener el mismo nombre pero pertenecer a contextos diferentes.

Por ejemplo: Una estructura de directorios hace que un archivo esté localizado en una carpeta o subcarpeta determinada, no pudiendo existir dos archivos con el mismo nombre en la misma.

Namespace: es como una “carpeta virtual” dentro del código PHP que permite agrupar clases relacionadas bajo un mismo nombre lógico.

Para definirla se usa la palabra clave **namespace** por ejemplo:

```
namespace App;
```

Veremos un ejemplo con el código **namespace_Persona.php**

Referencia: <https://www.php.net/manual/es/language.namespaces.php>



Programación en capas.

Composer

Composer es un gestor de dependencias en proyectos usado en la programación de aplicaciones PHP. Es el equivalente a maven en java.

Se utiliza para:

- Instalar y actualizar librerías de terceros.
- Autocargar tus propias clases.
- Organizar dependencias y versiones de forma automática.

Sus principales ventajas:

- Es simple de utilizar
- Cuenta con un repositorio propio.
- Disminuye significativamente problemas de cambio de ejecución.
- Actualiza las librerías a nuevas versiones de manera fácil.



Programación en capas.

Composer

Para instalar el **composer**:

- Descargar el instalador.
- Ejecutarlo y seleccionar la instalación de PHP que tienes instalada en tu ordenador.
- Abrir una consola (cmd) y ejecutar para verificar su instalación:

```
composer -- version
```

URL <https://getcomposer.org/download/>

Programación en capas.

Separación de la lógica de negocio

Motores de plantillas (template engines): es una aplicación que genera una página web a partir de un fichero con la información de presentación y otro con la lógica interna de la aplicación.

Facilitando la separación de la lógica del código HTML (la Vista del MVC) de una aplicación desarrollada.

Existen varios motores de plantillas:

- Twig (usado por Symfony).
- Blade (usado por Laravel).
- Smarty
- Plates
- Mustache

Programación en capas.

Separación de la lógica de negocio

Blade (template engines): es un sistema de plantillas que permite la generación de HTML dinámico con una sintaxis limpia.

Sus características son:

- Herencia de vistas (@extends, @section, @yield).
- Directivas personalizadas (@if, @foreach,...)
- Cachea las plantillas para un mejor rendimiento.
- Integrado totalmente con Laravel.

Fuente: <https://laravel.com/docs/blade>